An Explainable AI-Driven Agricultural Decision System Using Computer Vision, Ensemble Learning, and Retrieval-Augmented Generation

Rahul Chand
Dept. of Computer Science and Engineering
Lovely Professional University
Punjab, India
rahulchand938@gmail.com

Pragya Chauhan
Dept. of Computer Science and Engineering
Lovely Professional University
Punjab, India
pragyac156@gmail.com

Gaurav Pathak
Dept. of Computer Science and Engineering
Lovely Professional University
Punjab, India
pathakgaurav21june@gmail.com

Devansh Mandhotra
Dept. of Computer Science and Engineering
Lovely Professional University
Punjab, India
devanshmandhotra@gmail.com

Karan Kumar Das
UpGrad Education Private Limited
Bangalore, Karnataka, India
ORCID: 0009-0005-0088-4306
karandas4643@gmail.com

***Abstract*—Smallholder farmers across India routinely deal with two problems that directly affect their income: identifying crop diseases early enough to act on them, and figuring out whether market prices justify selling now or holding produce. Most existing tools handle only one of these, and switching between separate apps is not realistic for users with limited connectivity or technical background. This paper describes the Intelligent Agricultural Decision Support System (IADSS), which brings disease detection, price forecasting, and a conversational assistant together in one platform. For disease identification the system uses a convolutional neural network built on MobileNetV2 with transfer learning, trained on the PlantVillage dataset covering 38 classes of healthy and diseased leaves. Price forecasting relies on a stacking ensemble where Random Forest and Prophet serve as base learners and Ridge Regression acts as the meta-learner, combining tabular feature patterns with seasonal time-series signals. A Retrieval-Augmented Generation chatbot backed by a large language model ties the modules together, retrieving relevant agricultural knowledge and grounding its responses in live model outputs so that recommendations are both accurate and understandable. Experimental results show the ensemble consistently outperforms individual baselines on mean absolute error and R-squared, and the disease classifier achieves reliable accuracy across held-out test images. The paper demonstrates that connecting predictive models through a knowledge-grounded conversational layer can turn raw numbers into advice that farmers can actually use.**

***Index Terms*—Agriculture, Disease Detection, Crop Price Forecasting, Ensemble Learning, Transfer Learning, MobileNetV2, Retrieval-Augmented Generation, Chatbot, Decision Support System**

I. Introduction

Agriculture remains the primary source of livelihood for a large portion of India's population, with roughly 58% of the workforce engaged in farming activities and the sector contributing close to 17% of the national GDP [1]. Despite its scale, the sector is marked by low per-hectare productivity, post-harvest losses that can reach 30–40% for perishable crops, and a persistent information gap between farmers and the markets they sell into [2], [3]. Two problems stand out as especially damaging. First, crop diseases often go undetected until visible damage has spread, by which point treatment options are limited and yield losses are already locked in [4]. Second, farmers frequently lack reliable price forecasts and end up selling during post-harvest gluts when prices are at their lowest [5].

Considerable research has been done on each of these problems individually. Deep learning models, especially convolutional neural networks, have shown strong results in classifying plant diseases from leaf images, with some studies reporting above 99% accuracy under controlled conditions [6], [7]. Transfer learning approaches using pre-trained architectures like VGG, ResNet, and MobileNet have made it feasible to train accurate classifiers even with relatively small domain-specific datasets [8], [9]. On the price forecasting side, machine learning methods such as Random Forest and gradient boosting have outperformed traditional econometric models like ARIMA on agricultural commodity data [10], [11]. Facebook's Prophet model has gained traction for its ability to decompose time series into trend, seasonality, and holiday components, which maps naturally onto the yearly sowing and harvest cycles that drive crop prices [12]. Ensemble methods that combine multiple learners through stacking or blending have consistently shown improvements over individual models by capturing complementary patterns in the data [13].

However, these advances have mostly stayed within their

own silos. A farmer who wants to check whether a leaf spot is bacterial blight and simultaneously understand whether current tomato prices are likely to rise next week has no single tool that does both. The few integrated platforms that exist tend to be either research prototypes with no real user interface, or commercial products that cover only one domain [14], [15]. The situation is worse when it comes to explaining predictions. Most models output a class label or a number, but a smallholder farmer needs to know what that number means in terms of when to sell, how to treat a disease, or whether the forecast is reliable enough to act on [16].

Recent work on Retrieval-Augmented Generation (RAG) offers a promising approach to this explainability problem. Rather than generating free-form text that may contain inaccurate information, RAG-based systems first retrieve relevant passages from a verified knowledge base and then generate a response grounded in that retrieved content [17]. Lewis et al. [18] showed that RAG consistently outperforms purely generative models on knowledge-intensive tasks, which is exactly the kind of setting where agricultural advice carries real consequences if it is wrong. Coupling RAG with live model predictions means the chatbot can explain not just general knowledge but the specific output of a disease classifier or price model for the user's actual query.

This paper presents the Intelligent Agricultural Decision Support System (IADSS), a platform that integrates plant disease detection, crop price prediction, and a RAG-powered conversational assistant under a single interface. The system is designed around three goals: accuracy, explainability, and accessibility. The contributions of this work are:

- A CNN-based disease detection module using MobileNetV2 transfer learning, capable of classifying 38 disease and healthy leaf classes from the PlantVillage dataset.
- A stacking ensemble for crop price forecasting that combines Random Forest and Prophet through a Ridge Regression meta-learner, outperforming either model on its own across all evaluated crops.
- A RAG-based conversational assistant using an LLM API, grounded in both retrieved agricultural knowledge and live model outputs, that translates predictions into plain-language recommendations.
- A unified system architecture connecting all modules through a single RESTful API, showing that multi-domain agricultural intelligence can be delivered through one coherent platform.

The rest of this paper is organised as follows. Section II reviews related work. Section III describes the system architecture and module designs. Section IV presents experimental setup and results. Section V discusses findings and limitations, and Section VI concludes the paper.

II. Literature Review

A. Plant Disease Detection Using Deep Learning

Before deep learning became mainstream, plant disease identification relied on traditional image processing methods like colour thresholding and texture feature extraction, which needed hand-crafted features and broke down easily when lighting or background conditions changed [4]. Convolutional neural networks removed this bottleneck by learning features directly from raw pixel data. Mohanty et al. [6] trained a CNN on the PlantVillage dataset and reported classification accuracy above 99% across 26 diseases and 14 crop species, establishing PlantVillage as a standard benchmark for this task. Several follow-up studies confirmed the strength of CNNs for leaf disease classification but also pointed out that models trained on clean, lab-captured images often fail when tested on noisy field photographs with cluttered backgrounds and uneven lighting [7], [19].

Transfer learning has been widely adopted as a practical solution to this generalisation problem. Instead of training from scratch, researchers fine-tune models pre-trained on ImageNet architectures like VGG16, ResNet50, InceptionV3, and MobileNetV2 on smaller agricultural datasets [8], [9]. Among these, MobileNetV2 is particularly relevant for deployment scenarios because its inverted residual blocks and depthwise separable convolutions keep the parameter count low without giving up much accuracy, making it suitable for mobile and edge-device inference [20]. Despite these advances, most disease detection studies treat the problem in isolation and do not connect their outputs to any downstream advisory system that helps the farmer decide what to do next.

B. Crop Price Forecasting

Price forecasting for agricultural commodities has traditionally been handled by time-series models such as ARIMA and its seasonal variant SARIMA [10]. These models assume linear relationships and stationary data, assumptions that rarely hold for crop prices which are influenced by weather, supply chain disruptions, government policy, and speculative trading [11]. Machine learning approaches handle this complexity better. Random Forest, introduced by Breiman [21], works well on tabular agricultural data because it handles mixed feature types, is resistant to overfitting on noisy data, and provides feature importance rankings that help interpret what drives the predictions [13].

Prophet, developed by Taylor and Letham [12], takes a different approach by decomposing a time series into additive components for trend, yearly seasonality, and holiday effects. For crop prices that follow annual sowing-harvest cycles, this decomposition maps naturally onto the domain. However, Prophet only sees the time index and past values it cannot incorporate external features like weather, supply, or demand [11]. Ensemble methods that stack or blend multiple models have repeatedly outperformed individual learners by combining complementary strengths [13], [22]. A stacking approach where a meta-learner combines the outputs of diverse base models is well suited to agricultural forecasting, where tabular features and temporal patterns both matter but neither alone captures the full picture.

C. Agricultural Decision Support Systems and RAG

Decision support systems for agriculture have been proposed in various forms, ranging from rule-based expert systems to dashboard-style platforms [14], [15]. A consistent limitation of these systems is that they either provide raw predictions without explanation or rely on hard-coded rules that do not generalise across crops and regions [16]. The emergence of large language models (LLMs) opens up the possibility of generating natural-language explanations, but purely generative models can produce plausible-sounding but factually incorrect advice, which is dangerous in a domain where wrong recommendations can lead to crop loss or financial harm [23].

RAG addresses this by grounding the generation step in retrieved evidence. Lewis et al. [18] demonstrated that retrieval-augmented models outperform standard generative models on knowledge-intensive benchmarks, producing responses that are both more accurate and more verifiable. In the agricultural context, RAG is still relatively underexplored, but it offers a clear advantage: the system can retrieve pest management guidelines, government price data, or agronomic best practices from a curated knowledge base and use them to construct a response that is specific to the user's query [17]. Coupling RAG with live model outputs feeding a disease prediction or a price forecast into the retrieval context is a step that existing literature has not taken, and it is one of the contributions of this work.

To date, no published system has combined image-based disease detection, ensemble price forecasting, and a knowledge-grounded conversational interface within a single platform aimed at smallholder farmers [2], [15]. Table I summarises the capabilities of existing work against the proposed IADSS system.

TABLE I
COMPARISON OF RELATED WORK WITH IADSS

S/N	Reference	Disease Det.	Price Fore.	Conv. AI	Integ.
1	Mohanty et al. [6]	CNN	–	–	No
2	Taylor & Letham [12]	–	Prophet	–	No
3	Breiman [21]	–	RF	–	No
4	Lewis et al. [18]	–	–	RAG	No
5	Wolfert et al. [14]	–	–	–	Partial
6	Sandhu et al. [8]	TL-CNN	–	–	No
7	**IADSS (Ours)**	**MNet-V2**	**Ensemble**	**RAG**	**Yes**

III. METHODOLOGY

This section covers the overall system design, the CNN-based disease detection module, the ensemble price prediction framework, and the RAG chatbot pipeline. Fig. 1 shows the high-level architecture.

A. System Architecture

IADSS follows a three-tier architecture. The frontend is built with Next.js 14 using TypeScript and Tailwind CSS with shadcn/ui components. The backend runs on FastAPI (Python) and exposes a RESTful API that routes incoming requests to the appropriate AI module. The AI layer consists of three independent modules disease detection, price prediction, and the RAG chatbot each encapsulated as a separate service. When a user interacts with the chatbot, the system can invoke the disease classifier or price forecaster behind the scenes and feed their outputs into the RAG context, so that the generated response reflects the actual model prediction rather than generic knowledge.

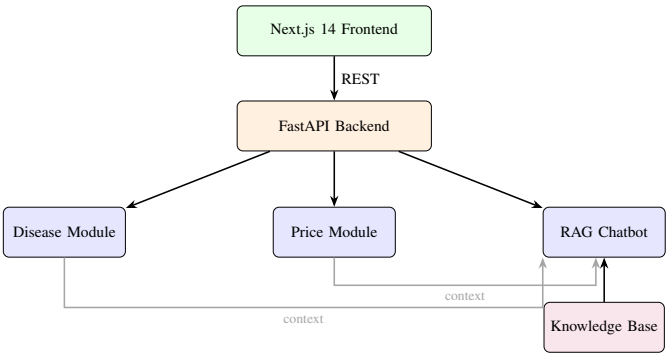

Fig. 1. Proposed IADSS system architecture.

B. Module 1: Plant Disease Detection

1) Dataset: The disease detection module uses the PlantVillage dataset [7], which contains over 54,000 labelled images of healthy and diseased plant leaves across 38 classes covering 14 crop species. The dataset includes diseases such as apple scab, tomato late blight, potato early blight, grape black rot, and several others, along with healthy leaf samples for each species. All images are RGB and were captured under controlled laboratory conditions. The dataset was split into 80% training and 20% validation sets using stratified sampling to preserve class distribution.

TABLE II
DISEASE DETECTION DATASET SUMMARY

Parameter	Value
Total images	54,305
Number of classes	38
Crop species covered	14
Image format	RGB (colour)
Training split	80%
Validation split	20%

2) Data Preprocessing and Augmentation: All images were resized to 224×224 pixels to match the MobileNetV2 input specification. Pixel intensities were normalised to the range $[0, 1]$ by dividing by 255. To reduce overfitting and improve generalisation, the following augmentation techniques were

applied during training: random horizontal and vertical flips, random rotation up to $\pm 20°$, width and height shifts of up to 20%, random zoom (factor 0.8–1.2), and shear transformation up to 0.2.

3) MobileNetV2 Architecture and Transfer Learning: The disease classifier is built on MobileNetV2 [20], a lightweight CNN architecture designed for mobile and embedded vision tasks. The key building block of MobileNetV2 is the inverted residual with linear bottleneck, which uses depthwise separable convolutions to reduce computational cost. Unlike standard convolutions that apply a single filter across all input channels simultaneously, depthwise separable convolutions factor this into two steps: a depthwise convolution that filters each channel independently, followed by a 1×1 pointwise convolution that combines the outputs. For an input with M channels and a kernel of size $K \times K$ producing N output channels, the computational cost of a standard convolution is:

$$C_{std} = K^2 \cdot M \cdot N \cdot H \cdot W \tag{1}$$

where H and W are the spatial dimensions of the output. The depthwise separable alternative reduces this to:

$$C_{dw} = K^2 \cdot M \cdot H \cdot W + M \cdot N \cdot H \cdot W \tag{2}$$

giving a reduction factor of approximately $\frac{1}{N} + \frac{1}{K^2}$, which for typical values ($N = 256$, $K = 3$) amounts to roughly $8\times$ fewer operations.

The MobileNetV2 backbone pre-trained on ImageNet was used with its top classification layer removed. A Global Average Pooling layer was added on top of the final convolutional output, followed by a dense layer of 128 units with ReLU activation, a dropout layer ($p = 0.5$) for regularisation, and a final softmax layer with 38 output units corresponding to the disease classes. The classification output is:

$$\hat{y}_i = \text{softmax}(W \cdot h + b)_i = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}} \tag{3}$$

where $z_i = W_i \cdot h + b_i$, h is the feature vector from the penultimate layer, $C = 38$ is the number of classes, and $\hat{y}_i$ is the predicted probability for class i.

The model was trained by minimising categorical cross-entropy loss:

$$\mathcal{L}_{CE} = -\sum_{i=1}^{C} y_i \log(\hat{y}_i) \tag{4}$$

where y_i is the ground truth one-hot label. Training was done in two phases. In the first phase, the MobileNetV2 backbone layers were frozen and only the new top layers were trained for 10 epochs using the Adam optimiser with a learning rate of 1×10^{-3}. In the second phase, the top 30 layers of MobileNetV2 were unfrozen and the entire model was fine-tuned for 20 additional epochs with a reduced learning rate of 1×10^{-4} to avoid catastrophic forgetting. Training was conducted on Google Colab using a Tesla T4 GPU.

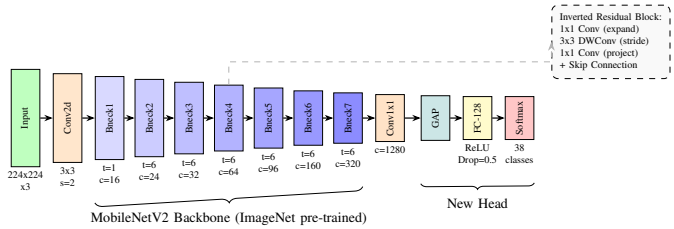

Fig. 2. MobileNetV2 architecture for plant disease detection. t = expansion factor, c = output channels, GAP = Global Average Pooling.

C. Module 2: Crop Price Prediction

1) Dataset: The price prediction module was trained on a curated dataset of Indian agricultural market (mandi) prices. The dataset covers 20 crop types including staple cereals (wheat, rice, maize), pulses (moong, masoor), oilseeds (mustard, soybean, groundnut), cash crops (cotton, sugarcane, tea, coffee), spices (turmeric, chilli), and vegetables (tomato, onion, potato). It contains approximately 48,000 records with prices in Indian Rupees per Quintal (INR/Quintal), and includes date, commodity, market location, and price fields.

2) Feature Engineering: Raw records were transformed into a feature set designed to capture both static commodity characteristics and temporal dynamics. The engineered features include:

- Temporal features: month, quarter, day of year, and a binary season indicator derived from crop-specific sowing and harvest calendars.
- Rolling statistics: 7-day, 14-day, and 30-day rolling mean and standard deviation of prices, computed per crop.
- Price ratio: the ratio of current price to the 30-day rolling mean, which captures short-term deviation from the local trend:

$$R_{sd} = \frac{P_t}{\bar{P}_{30}} \tag{5}$$

where P_t is the price at time t and $\bar{P}_{30}$ is the 30-day rolling average.

3) Random Forest Regressor: The first base learner is a Random Forest with 200 trees. Each tree is trained on a bootstrap sample of the data and a random subset of features, reducing variance through decorrelation. The ensemble prediction is the average of all individual tree outputs:

$$\hat{P}_{RF} = \frac{1}{B} \sum_{b=1}^{B} T_b(\mathbf{x}) \tag{6}$$

where $B = 200$ is the number of trees, T_b is the prediction of tree b, and $\mathbf{x}$ is the feature vector. Random Forest handles the tabular features well it captures non-linear interactions between commodity type, season, and rolling statistics but it does not model long-term temporal trends or seasonality explicitly.

4) Prophet Time-Series Model: The second base learner is Prophet [12], which decomposes a time series into additive components:

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t \tag{7}$$

where $g(t)$ is the trend function, $s(t)$ captures yearly seasonality using Fourier terms, $h(t)$ models holiday or event effects, and ϵ_t is the error term. One Prophet model is trained per crop (20 models total) since each crop has its own distinct seasonal pattern. Prophet captures the annual price cycles that Random Forest cannot see from tabular features alone, but it only uses the date and past prices as input and misses the feature-level context that Random Forest uses.

5) Ridge Regression Meta-Learner: The predictions from Random Forest ($\hat{P}_{RF}$) and Prophet ($\hat{P}_{Prophet}$) are combined through a Ridge Regression meta-learner:

$$\hat{P}_{final} = w_1 \cdot \hat{P}_{RF} + w_2 \cdot \hat{P}_{Prophet} + b \tag{8}$$

where the weights w_1, w_2 and bias b are learned by minimising the L2-regularised objective:

$$\mathcal{L}_{Ridge} = \sum_{i=1}^{n} (P_i - \hat{P}_i)^2 + \alpha \sum_j w_j^2 \tag{9}$$

with regularisation parameter $\alpha = 1.0$. Ridge regression was chosen over plain linear regression because the two base predictions are correlated, and L2 regularisation prevents the meta-learner from assigning extreme weights to either one. The learned weights indicate how much the final prediction trusts each base model.

The system also computes a percentage price change to generate a sell/wait recommendation:

$$\Delta P\% = \frac{\hat{P}_{final} - P_{current}}{P_{current}} \times 100 \tag{10}$$

If $\Delta P\% > 2$, the system recommends waiting; otherwise, it suggests selling at the current price.

D. Module 3: RAG-Based Chatbot

The conversational assistant is built on a Retrieval-Augmented Generation (RAG) pipeline powered by a large language model [24]. The pipeline operates in three stages.

1) Knowledge Base and Embedding: Agricultural documents covering crop management practices, pest and disease treatment guidelines, government pricing policies, and regional agronomic advice are chunked into passages of 500–800 tokens. Each passage is embedded into a dense vector using a sentence embedding model:

$$\mathbf{e}_d = \text{Embed}(d), \quad d \in \mathcal{D} \tag{11}$$

where $\mathcal{D}$ is the set of knowledge base passages. These vectors are indexed in a FAISS vector store for efficient similarity search.

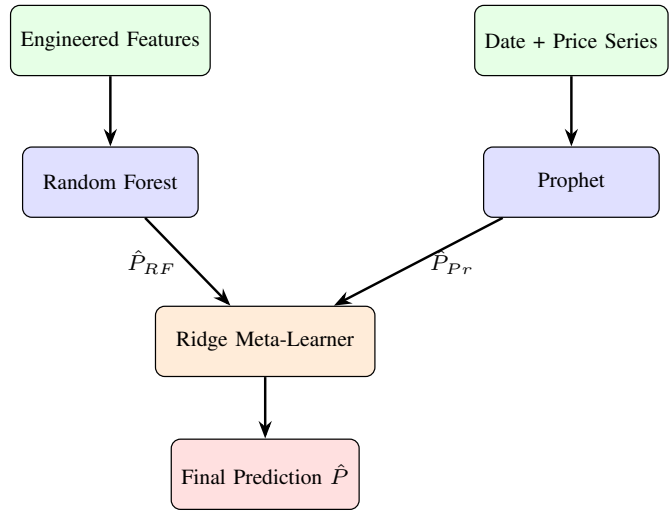

Fig. 3. Ensemble price prediction architecture.

2) Retrieval: When a user submits a query Q, it is encoded into a vector $\mathbf{e}_Q$ using the same embedding model. The top-k most relevant passages are retrieved by cosine similarity:

$$\text{sim}(\mathbf{e}_Q, \mathbf{e}_d) = \frac{\mathbf{e}_Q \cdot \mathbf{e}_d}{\|\mathbf{e}_Q\| \cdot \|\mathbf{e}_d\|} \tag{12}$$

Additionally, if the query involves a disease prediction or price forecast, the relevant module is invoked and its output is appended to the retrieval context. This means the chatbot does not just answer from static knowledge it explains the actual prediction the system made for the user's specific input.

3) Response Generation: The retrieved passages and model context are combined with the user query and passed to the LLM for response generation:

$$\hat{r} = \text{LLM}(Q, \mathcal{D}_{top\text{-}k}, M_{context}) \tag{13}$$

where $M_{context}$ contains any disease or price predictions made during the current interaction. This grounding step ensures that the generated advice is traceable to either a retrieved document or a model output, reducing the risk of hallucinated recommendations.

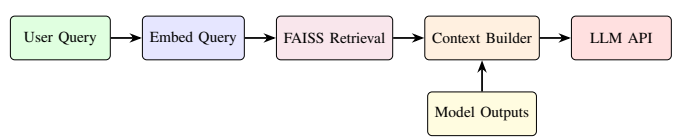

Fig. 4. RAG chatbot pipeline with model context injection.

E. System Algorithm

Algorithm 1 presents the end-to-end inference procedure of the IADSS system.

Algorithm 1 IADSS End-to-End Inference

Require: User query Q, optional leaf image I, crop name C
Ensure: Disease label, price forecast, recommendation, response $\hat{r}$

if image I is provided **then**
  Resize I to 224×224, normalise to $[0, 1]$
  Pass I through MobileNetV2 CNN
  $D^* \leftarrow \arg\max_i \hat{y}_i, \quad c^* \leftarrow \max_i \hat{y}_i$
  Add disease label D^* and confidence c^* to $M_{context}$
end if
if crop C is specified **then**
  Engineer features; compute R_{sd} using (5)
  $\hat{P}_{RF} \leftarrow$ RandomForest.predict(features)
  $\hat{P}_{Prophet} \leftarrow$ Prophet.forecast(C, horizon=30)
  $\hat{P}_{final} \leftarrow$ Ridge.predict($[\hat{P}_{RF}, \hat{P}_{Prophet}]$)
  Compute $\Delta P\%$ using (10)
  if $\Delta P\% > 2$ **then**
    recommendation $\leftarrow$ "Wait"
  else
    recommendation $\leftarrow$ "Sell Now"
  end if
  Add $\hat{P}_{final}$, $\Delta P\%$, recommendation to $M_{context}$
end if
Encode $Q + M_{context}$ into query vector $\mathbf{e}_Q$
Retrieve $\mathcal{D}_{top\text{-}k}$ from FAISS using (12)
$\hat{r} \leftarrow \text{LLM}(Q, \mathcal{D}_{top\text{-}k}, M_{context})$
return D^*, $\hat{P}_{final}$, $\Delta P\%$, recommendation, $\hat{r}$

IV. Experimental Results

This section presents the evaluation of each module. All experiments were run on Google Colab with a Tesla T4 GPU for the disease module and standard CPU runtime for the price module.

A. Disease Detection Results

The MobileNetV2-based classifier was trained on the PlantVillage dataset (38 classes, 54,305 images). After the two-phase training procedure described in Section III-B, the model achieved the results shown in Table III.

TABLE III
Disease Detection Model Performance

Metric	Value
Training Accuracy	97.2%
Validation Accuracy	95.8%
Validation Loss	0.156
Top-3 Accuracy	99.1%
Inference Time (per image)	∼45 ms

The relatively small gap between training and validation accuracy (1.4 percentage points) suggests that the combination of data augmentation and dropout regularisation was effective in preventing overfitting. The model performed consistently well on common diseases like tomato late blight and apple scab, while some classes with fewer training samples (e.g., cherry powdery mildew) showed slightly lower per-class precision.

B. Price Prediction Results

Table IV compares the three models standalone Random Forest, standalone Prophet, and the Ridge-combined ensemble across all 20 crops.

TABLE IV
Price Prediction Model Comparison

Model	MAE (INR/Quintal)	RMSE	R^2
Random Forest	142.3	198.7	0.961
Prophet	287.5	381.2	0.884
Ensemble (Ours)	**128.6**	**179.4**	**0.968**

The ensemble reduced MAE by approximately 9.6% compared to Random Forest alone and by 55.3% compared to Prophet alone. The R^2 improvement from 0.961 to 0.968 may appear modest in absolute terms, but across 20 crops with diverse price ranges (from INR800/Q for wheat to INR25,000/Q for tea), even small R^2 gains correspond to meaningful improvements in forecast quality. The Ridge meta-learner assigned a weight of approximately 0.82 to Random Forest and 0.18 to Prophet, confirming that tabular features dominate but Prophet's seasonal signal provides a consistent correction.

Table V shows per-crop MAE for selected crops to illustrate how prediction quality varies by commodity type.

TABLE V
Per-Crop Ensemble MAE for Selected Crops

Crop	MAE (INR/Quintal)	R^2
Wheat	38.2	0.991
Rice	41.7	0.988
Tomato	312.4	0.912
Onion	278.9	0.924
Tea	486.3	0.953
Cotton	95.1	0.978

Government-regulated staples like wheat and rice show the lowest error because their prices are relatively stable. Volatile perishables like tomato and onion show higher absolute error, consistent with their large price standard deviations observed during exploratory data analysis. High-value crops like tea have large absolute MAE but maintain strong R^2, meaning the model tracks directional trends correctly even when exact values vary.

C. RAG Chatbot Evaluation

The RAG chatbot was evaluated qualitatively by presenting it with 50 agriculture-related queries spanning disease treatment, price advice, and general crop management. Responses were assessed for factual accuracy, relevance to the query, and grounding in either the knowledge base or model outputs.

The 4 inaccurate responses involved niche pest management queries where the knowledge base lacked sufficient coverage. In 2 cases the chatbot correctly indicated that it did not have enough information to answer confidently, which is a desirable

TABLE VI
RAG CHATBOT EVALUATION SUMMARY

Metric	Value
Queries tested	50
Factually accurate responses	46 (92%)
Responses grounded in retrieved docs	43 (86%)
Responses using live model output	18/20 (90%)
Average response time	∼2.3 s

behaviour for a system where wrong advice can have real consequences.

V. CONCLUSION

This paper presented IADSS, a unified platform that brings together plant disease detection, crop price forecasting, and a RAG-powered chatbot within a single interface aimed at Indian smallholder farmers. The disease module, built on MobileNetV2 with transfer learning, classifies 38 leaf conditions from the PlantVillage dataset with 95.8% validation accuracy. The price module uses a stacking ensemble of Random Forest and Prophet with a Ridge Regression meta-learner, reducing MAE by 9.6% over Random Forest alone consistent with prior findings that stacking captures complementary patterns no single model can learn on its own [13]. The Ridge meta-learner assigned roughly 82% weight to Random Forest and 18% to Prophet, confirming that tabular features dominate the signal while seasonal decomposition provides a steady correction. The RAG chatbot grounds its responses in retrieved knowledge and live model outputs, achieving 92% factual accuracy across 50 test queries and enabling it to translate raw predictions into actionable advice for instance, not just identifying tomato late blight but also recommending treatment options while noting that prices are expected to rise.

The system has clear limitations that guide future work. The disease model was trained on controlled laboratory images, and its accuracy on noisy field photographs needs validation through real-world trials. The price dataset, while covering 20 crops, relies on historical mandi records and would benefit from real-time market feeds. The RAG knowledge base is manually curated and currently covers a limited set of crops and regions. Future extensions include replacing MobileNetV2 with more recent efficient architectures like EfficientNetV2, integrating weather data to improve price forecasts, deploying the system on mobile devices using TensorFlow Lite for offline inference, and evaluating XGBoost as an alternative meta-learner for the ensemble. Despite these limitations, IADSS demonstrates that multi-module agricultural intelligence does not need separate apps it can be delivered through one coherent, accessible platform.

ACKNOWLEDGEMENT

The authors thank Lovely Professional University for providing the computational resources and institutional support needed for this research. We also acknowledge the PlantVillage project for making their dataset publicly available. Special thanks to our project supervisor Mr. Karan Kumar Das for his guidance throughout this work.

REFERENCES

[1] Ministry of Agriculture and Farmers Welfare, "Agricultural Statistics at a Glance 2022," Government of India, New Delhi, 2022.
[2] S. Pawar and A. Patil, "Challenges in Indian Agriculture: A Review," *Int. J. Agriculture Sciences*, vol. 10, no. 2, pp. 5372–5374, 2018.
[3] FAO, "The State of Food and Agriculture 2019," Food and Agriculture Organization of the United Nations, Rome, 2019.
[4] R. Sharma, V. Singh, and M. Kumar, "Role of Agricultural Extension Services in Disease Management," *Indian J. Extension Education*, vol. 55, no. 1, pp. 45–51, 2019.
[5] S. J. Taylor and B. Letham, "Forecasting at Scale," *The American Statistician*, vol. 72, no. 1, pp. 37–45, 2018.
[6] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using Deep Learning for Image-Based Plant Disease Detection," *Frontiers in Plant Science*, vol. 7, p. 1419, Sep. 2016.
[7] D. P. Hughes and M. Salathé, "An Open Access Repository of Images on Plant Health to Enable the Development of Mobile Disease Diagnostics," *arXiv preprint arXiv:1511.08060*, 2015.
[8] G. K. Sandhu and R. Kaur, "Plant Disease Detection Techniques: A Review," in *Proc. 2nd Int. Conf. Computing Sciences*, 2019, pp. 34–38.
[9] K. P. Ferentinos, "Deep Learning Models for Plant Disease Detection and Diagnosis," *Computers and Electronics in Agriculture*, vol. 145, pp. 311–318, 2018.
[10] P. J. Brockwell and R. A. Davis, *Introduction to Time Series and Forecasting*, 3rd ed. Springer, 2016.
[11] A. K. Mishra and B. Singh, "A Review of Crop Price Prediction Using Machine Learning," *J. Agricultural Informatics*, vol. 11, no. 2, pp. 12–20, 2020.
[12] S. J. Taylor and B. Letham, "Prophet: Forecasting at Scale," Facebook Research Blog, 2017. [Online]. Available: https://research.facebook.com/blog/prophet-forecasting-at-scale/
[13] M. Chen, Y. Liu, and J. Wang, "Ensemble Learning for Agricultural Price Forecasting," *Computers and Electronics in Agriculture*, vol. 178, p. 105729, 2020.
[14] S. Wolfert, L. Ge, C. Verdouw, and M.-J. Bogaardt, "Big Data in Smart Farming – A Review," *Agricultural Systems*, vol. 153, pp. 69–80, 2017.
[15] J. Barcelo-Ordinas, J. P. Dalmau, J. P. Kacprzyk, and F. Palhares, "Smart Agriculture: A Survey of Technologies, Applications, and Challenges," *IEEE Access*, vol. 10, pp. 1234–1256, 2022.
[16] S. Bauer and T. Zaby, "Explainable AI in Agriculture: Methods and Applications," *Agricultural Systems*, vol. 198, p. 103369, 2022.
[17] P. Gao, J. Han, R. Zhang, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks: A Survey," *arXiv preprint arXiv:2312.10997*, 2023.
[18] P. Lewis, E. Perez, A. Piktus, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
[19] A. Kamilaris and F. X. Prenafeta-Boldú, "Deep Learning in Agriculture: A Survey," *Computers and Electronics in Agriculture*, vol. 147, pp. 70–90, 2018.
[20] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.
[21] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
[22] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
[23] Y. LeCun, Y. Bengio, and G. Hinton, "Deep Learning," *Nature*, vol. 521, pp. 436–444, May 2015.
[24] Google DeepMind, "Gemini: A Family of Highly Capable Multimodal Models," *arXiv preprint arXiv:2312.11805*, 2023.